

Linux Server Operations Lab

Phase 8 レビュー

Excelを「台帳」から「操作盤」へ - Excel × Python × Ansible × Linux

Excel	→	Python	→	Ansible	→	Linux	→	Excel
作業指示		解析・制御		実行		変更		結果記録

Phase 8の目的

Phase 7ではLinuxの状態を取得してExcelへ書き戻しました。Phase 8では逆方向の処理を作り、Excelに書いた作業指示をPythonが読み、Ansibleを通してLinuxへ反映し、実行結果を再びExcelへ記録する仕組みを構築しました。

Excelが単なる管理台帳ではなく、作業依頼と実行結果を扱う簡易操作盤として機能する状態まで作成しました。

1. 03_OPERATION_REQUESTS シートの作成

操作指示専用のシートを追加し、以下の7列を用意しました。

列	項目	役割
A	RequestID	作業依頼を一意に識別
B	ServerID	対象サーバを指定
C	Action	実行する操作
D	Execute	YES / NOで実行可否を指定
E	Status	SUCCESS / FAILEDを記録
F	Message	実行結果やエラー内容を記録
G	ExecutedAt	実行日時を記録

入力規則（プルダウン）

Action列には CREATE_TEST_FILE / DELETE_TEST_FILE / RESTART_NGINX、Execute列には YES / NO のプルダウンを設定しました。これにより表記揺れを防ぎ、Python側の完全一致判定での入力ミスを減らせます。

ServerIDだけを操作指示シートへ書き、HostnameやIPアドレスを重複管理しない構成にしました。サーバ情報の正本は01_SERVER_MASTERのままです。

2. Pythonによる操作指示の読み取り

execute_operations.py を作成し、Excelの内容から対象サーバと操作内容を決定するようにしました。

ServerID → Hostname の対応表

01_SERVER_MASTERを読み、SV001 → tokyo、SV002 → yokohama、SV003 → omiya の対応表をPython内で作成します。03_OPERATION_REQUESTS側ではServerIDだけ指定すれば対象ホストを特定できます。

実行条件と再実行防止

ExecuteがYESの行だけを処理し、StatusがSUCCESSの行はスキップします。これにより、同じ作業依頼の重複実行を防ぎました。

```
if execute != "YES":
    continue

if status == "SUCCESS":
    continue
```

対応したAction

Action	Linux側で行う処理
CREATE_TEST_FILE	/tmp/phase8_test.txt を作成
DELETE_TEST_FILE	/tmp/phase8_test.txt を削除
RESTART_NGINX	nginxサービスを再起動

3. 実行結果

3つの操作をExcelから指示し、最終的にすべてSUCCESSまで確認しました。

RequestID	ServerID	Action	結果
REQ001	SV001	CREATE_TEST_FILE	SUCCESS
REQ002	SV001	DELETE_TEST_FILE	SUCCESS
REQ003	SV001	RESTART_NGINX	SUCCESS

	A	B	C	D	E	F	G
1	RequestID	ServerID	Action	Execute	Status	Message	ExecutedAt
2	REQ001	SV001	CREATE_TEST_FILE	YES	SUCCESS	CREATE_TEST_FILE completed	2026-09-21 02:24:04
3	REQ002	SV001	DELETE_TEST_FILE	YES	SUCCESS	DELETE_TEST_FILE completed	2026-09-21 02:42:32
4	REQ003	SV001	RESTART_NGINX	YES	SUCCESS	RESTART_NGINX completed	2026-09-21 02:51:52
5							

図：03_OPERATION_REQUESTS

の最終状態。REQ001～REQ003がSUCCESSとなり、MessageとExecutedAtまで自動記録されています。

再実行防止の確認

全行がSUCCESSの状態でもう一度 execute_operations.py を実行したところ、Executing ...

は表示されず、Operation processing completed

のみが表示されました。SUCCESS済みの依頼が正しくスキップされています。

4. トラブルシュート

RESTART_NGINXだけ最初はFAILEDとなりました。原因はnginx再起動にsudo権限が必要だったためです。

発生した流れ

1	CREATE_TEST_FILE / DELETE_TEST_FILE一般ユーザー権限で成功	
2	RESTART_NGINX	sudo権限が必要でFAILED
3	原因切り分け	サービス操作だけ権限昇格が必要と確認
4	対策	nginx restart のみNOPASSWDで許可
5	再実行	REQ003もSUCCESS

sudoersで許可した操作

```
tokyo ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx
```

Pythonへsudoパスワードを直接書き込まず、許可するコマンドをnginx再起動だけに限定しました。検証環境で「必要な操作だけを許可する」という権限設計を学ぶ題材になりました。

その他の異常系

存在しないServerIDはFAILED + ServerID not found、未対応ActionはFAILED + Unknown actionとしてExcelへ記録するようにしています。

5. Phase 8で学んだこと

1. Excelは結果を見るだけの台帳ではなく、人が入力する操作インターフェースとしても使える。
2. 01_SERVER_MASTERを正本にして、操作指示側ではServerIDだけを使うことで重複管理を減らせる。
3. プルダウンによる入力規則は、Pythonに渡すデータ品質をExcel側で守る仕組みになる。
4. SUCCESS済みの依頼を再実行しない制御は、変更系自動化では重要な安全策になる。
5. sudoが必要な操作では、パスワード埋め込みではなく、必要最小限の権限設計を考える必要がある。
6. 処理結果と実行日時をExcelへ返すことで、人が後から確認できる運用記録になる。

Phase 8の完成形

Excel作業指示 → Python判定 → Ansible実行 → Linux変更 →
ExcelへSUCCESS/FAILEDと実行日時を書き戻す、という双方向の運用フローを構築できました。

6. 次のPhase

Phase

9では、意図的に障害や異常状態を作り、検知・切り分け・復旧までを一連の運用として検証します。Phase 5～8で作った点検・判定・操作・結果記録の仕組みを、障害対応へつなげる段階です。

note記事候補

- ExcelをAnsibleの操作盤にしてみた
- Excel → Python → Ansible → Linuxをつないで作業指示を自動化
- SUCCESS済みを再実行しない仕組みを入れた理由
- sudoパスワードをPythonへ書かずにnginx再起動を自動化した話